

Chapter 8: Arrays & Lists

By Irina Galata

As discussed in the introduction to this section, collections are flexible “containers” that let you store any number of values together. Two of the most common collections types in Kotlin are **arrays** and **lists**.

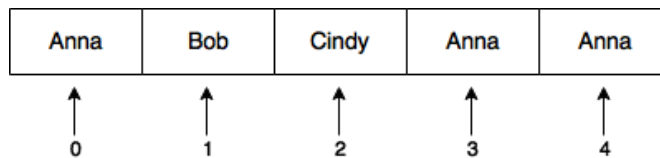
Arrays

Arrays in Kotlin correspond to the basic array type available in Java. Arrays are typed, just like regular variables and constants, and store multiple values in a contiguous portion of memory.

Before you create your first array, take some time to consider in detail what an array is and why you might want to use one.

What is an array?

An array is an ordered collection of values of the same type. The elements in the array are **zero-indexed**, which means the index of the first element is 0, the index of the second element is 1, and so on. Knowing this, you can work out that the last element's index is the number of values in the array less 1.



There are five elements in this array, at indices 0–4.

All values are of type `String`, so you can't add non-string types to an array that holds strings. Notice that the same value can appear multiple times.

When are arrays useful?

Arrays are useful when you want to store your items in a particular order. You may want the elements sorted, or you may need to fetch elements by index without iterating through the entire array.

For example, if you were storing high score data, then order would matter. You would want the highest score to come first in the list (i.e. at index 0) with the next-highest score after that, and so on.

Creating arrays

The easiest way to create an array is by using a function from the Kotlin standard library, `arrayOf()`. This is a concise way to provide array values.

```
val evenNumbers = arrayOf(2, 4, 6, 8)
```

Since the array only contains integers, Kotlin infers the type of `evenNumbers` to be an array of `Int` values. This type is written as `Array<Int>`. The type inside the angle brackets defines the type of values the array can store, which the compiler will enforce when you add elements to the array. If you try to add a string, for example, the compiler will return an error and your code won't compile. This syntax for the array type is an example of a **type argument** or **generic**, which you'll learn more about in a later chapter.

It's also possible to create an array with all of its values set to a default value:

```
val fiveFives = Array(5, { 5 }) // 5, 5, 5, 5, 5
```

You'll learn more about the `{ 5 }` syntax in the chapter on **lambdas**.

As with any type, it's good practice to declare arrays that aren't going to change as constants using `val`. For example, consider this array:

```
val vowels = arrayOf("a", "e", "i", "o", "u")
```

`vowels` is an array of strings and its values can't be changed. But that's fine, since the list of vowels doesn't tend to change very often!

Arrays of primitive types

When using `arrayOf()` and creating arrays with types such as `Array<Int>`, the resulting array is a list of object types. In particular, if you're running on the JVM, the integer type will be the boxed `Integer` class and not the primitive `int` type. Using primitive types over their boxed counterparts will consume less memory and result in better performance. Unfortunately you can't use primitives with lists (covered in the next section), so it will be up to you to determine on a case by case basis if the trade off is worth it!

The Kotlin standard library contains functions other than `arrayOf()` that make it possible to create arrays that correspond to arrays of primitive types. For example, you can create an array of odd numbers as follows:

```
val oddNumbers = intArrayOf(1, 3, 5, 7)
```

When running Kotlin on the JVM, the `oddNumbers` array is compiled to a Java array of type `int[]`.

Other standard library functions include `floatArrayOf()`, `doubleArrayOf()`, and `booleanArrayOf()`. These various functions create arrays of type `IntArray`, `FloatArray`, `DoubleArray`, etc. You can also pass a number into the constructor for these types, for example, to create an array of zeros.

```
val zeros = DoubleArray(4) // 0.0, 0.0, 0.0, 0.0
```

You can convert between the boxed and primitive arrays using functions like `toIntArray()`.

```
val otherOddNumbers = arrayOf(1, 3, 5, 7).toIntArray()
```

The type of `otherOddNumbers` is `IntArray` and not `Array<Int>`.

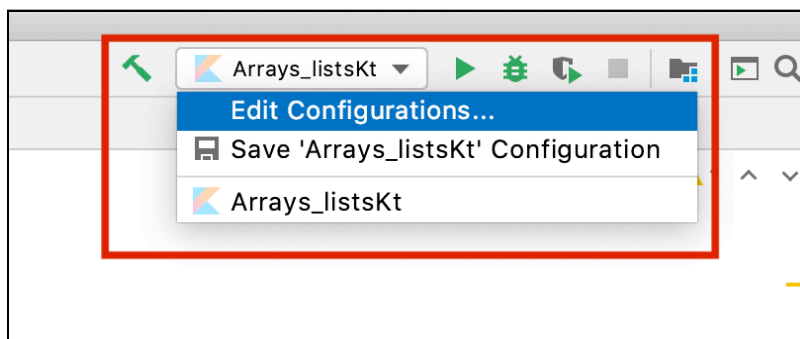
Arguments to main()

The `main()` function is the entry point to Kotlin programs. From Kotlin 1.3 onwards, the `main()` function has an optional parameter named `args` that is an `Array<String>`:

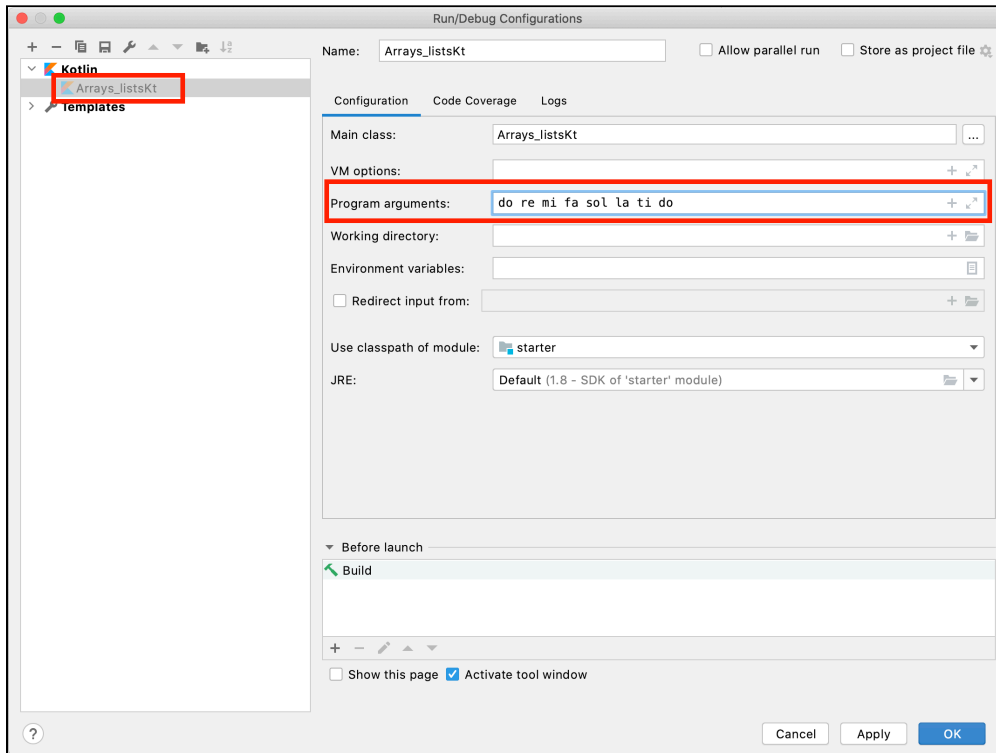
```
fun main(args: Array<String>) {  
}
```

When running a Kotlin program from the command-line, you can send arguments to `main()` like you would a typical command-line program.

Since we're using IntelliJ IDEA in the book, you can send arguments to `main()` using the project configuration, accessed via the **Edit Configurations...** menu in the IntelliJ toolbar:



This will pop up the **Run/Debug Configurations** window. Make sure your configuration is selected in the panel on the left, then add arguments to the **Program arguments** field on the right and click **OK**:



Iterating over an array

To see the arguments passed to `main()`, you can use the `for` loop you read about in Chapter 5, “Advanced Control Flow”.

However, for iterating over an array, instead of using a count and a range in the `for` loop, you give a name like `arg` to each element of the array args:

```
for (arg in args) {  
    println(arg)  
}  
// do  
// re  
// mi  
// fa  
// sol  
// la
```

```
// ti  
// do
```

The value of `arg` is updated for each iteration of the loop. There is one iteration of the loop for each element in the array `args`. Using `println(arg)` in the loop body prints each of the arguments passed to the `main()` function.

An alternative form of iteration uses `forEach` as a call on the array:

```
args.forEach { arg ->  
    println(arg)  
}
```

You'll learn more about the syntax used in the call to `forEach`, called *trailing lambda* syntax, in Chapter 10, "Lambdas".

Lists

A type that is very similar conceptually to an array is a **list**. Like in Java, the **List** type in Kotlin is an **interface** that has concrete realizations in types such as `ArrayList`, `LinkedList` and others. Arrays are typically more efficient than lists in terms of raw performance, but lists have the additional feature of being **dynamically-sized**. That is, arrays are of fixed-size, but lists can be setup to grow and shrink as needed, as you'll see later when learning about mutable lists.

Creating lists

Like with arrays, the standard library has a function to create a list.

```
val innerPlanets = listOf("Mercury", "Venus", "Earth", "Mars")
```

The type of `innerPlanets` is inferred to be `List<String>`, with `String` being another example of a type argument. So `innerPlanets` can be passed into any function that needs a `List`. Under the hood, the type used to store `innerPlanets` is an `ArrayList`. If, for some reason, you explicitly want `innerPlanets` to have the type `ArrayList`, there is a different standard library function you can use:

```
val innerPlanetsArrayList =  
    arrayListOf("Mercury", "Venus", "Earth", "Mars")
```

An empty list can be created by passing no arguments into `list()`. Because the compiler isn't able to infer a type from this, you need to use a type declaration to make the type explicit:

```
val subscribers: List<String> = listOf()
```

You could also put the the type argument on the function:

```
val subscribers = listOf<String>()
```

Since the list returned from `listOf()` is immutable, you won't be able to do much with this empty list. Empty lists become more useful as a starting point for a list when they're **mutable**.

None of `innerPlanets`, `innerPlanetsArrayList` or `subscribers` can be altered once created. For that, you'll need to instead create a mutable list.

Mutable lists

Once again, the standard library has a function to use here.

```
val outerPlanets =  
    mutableListOf("Jupiter", "Saturn", "Uranus", "Neptune")
```

You've made `outerPlanets` a mutable list, just in case Planet X is ever discovered in the outer solar system. You can create an empty mutable list by passing no arguments to the function:

```
val exoPlanets = mutableListOf<String>()
```

You'll see later in the chapter how to add and remove elements from a mutable list.

Accessing elements

Being able to create arrays and lists is useless unless you know how to fetch values from them. In this section, you'll learn several different ways to access the elements. The syntax is similar for both arrays and lists.

Using properties and methods

Imagine you're creating a game of cards, and you want to store the players' names in a list. The list will need to change as players join or leave the game, so you need to declare a mutable list:

```
val players = mutableListOf("Alice", "Bob", "Cindy", "Dan")
```

In this example, `players` is a mutable list because you used the `mutableListOf()` standard library function.

Before the game starts, you need to make sure there are enough players. You can use the `isEmpty()` *method* to check if there's at least one player:

```
print(players.isEmpty())  
// > false
```

The list isn't empty, but you need at least two players to start a game. You can get the number of players using the `size` *property*:

```
if (players.size < 2) {  
    println("We need at least two players!")  
} else {  
    println("Let's start!")  
}  
// > Let's start!
```

Note: You'll learn all about properties and methods in Chapter 11, "Classes," and even more in Chapters 13 and 14. For now, just think of properties as variables that are built into values. To access a property, place a dot after the name of the constant or variable that holds the value and follow it by the name of the property you want to access. Similarly, think of methods as functions that are built in to values.

It's time to start the game! You decide that the order of play is by the order of names in the list. How would you get the first player's name?

Lists provide the `first()` *method* to fetch the first object of a list:

```
var currentPlayer = players.first()
```


Printing the value of `currentPlayer` reveals an interesting question:

```
println(currentPlayer) // > Alice
```

What would be printed if the `players` list were empty? It turns out trying to do so will throw an exception, so be careful when using some of these properties and methods on lists!

Similarly, lists have a `last()` method that returns the last value in a list, or throws an exception if the list is empty:

```
println(players.last()) // > Dan
```

Another way to get values from a list is by calling `minOrNull()`. This method returns the element with the lowest *value* in the list — not the lowest index!

If the array contained strings, then it would return the string that's the lowest in alphabetical order, which in this case is "Alice":

```
val minPlayer = players.minOrNull()
minPlayer.let {
    println("$minPlayer will start") // > Alice will start
}
```

Instead of throwing an exception if no minimum can be determined, `minOrNull()` returns a nullable type, so you need to check if the value returned is null.

Obviously, `first()` and `minOrNull()` will not always return the same value. For example:

```
println(arrayOf(2, 3, 1).first())
// > 2
println(arrayOf(2, 3, 1).min())
// > 1
```

As you might have guessed, lists also have a `maxOrNull()` method.

```
val maxPlayer = players.maxOrNull()
if (maxPlayer != null) {
    println("$maxPlayer is the MAX") // > Dan is the MAX
}
```

Note: The `size` property and the `first()`, `last()`, `minOrNull()` and `maxOrNull()` methods aren't unique to arrays or lists. Every collection type has such properties and methods, in addition to a plethora of others. You'll learn more about this behavior when you read about interfaces in Chapter 17, "Interfaces."

The methods seen so far are helpful if you want to get the first, last, minimum or maximum elements. But what if the element you want can't be obtained with one of these methods?

Using indexing

The most convenient way to access elements in an array or list is by using the indexing syntax. This syntax lets you access any value directly by using its index inside square brackets:

```
val firstPlayer = players[0]
println("First player is $firstPlayer")
// > First player is Alice
```

Because arrays and lists are zero-indexed, you use index 0 to fetch the first object.

The indexing syntax is equivalent to calling `get()` on the array or list and passing in the index as an argument.

```
val secondPlayer = players.get(1)
```

You can use a greater index to get the next elements in the array or list, but if you try to access an index that's beyond the size of the array or list, you'll get a runtime error.

```
val player = players[4] // > IndexOutOfBoundsException
```

You receive this error because `players` contains only four strings. Index 4 represents the fifth element, but there is no fifth element in this list.

Using ranges to slice

You can use the `slice()` method with ranges to fetch more than a single value from an array or list.

For example, if you'd like to get the next two players, you could do this:

```
val upcomingPlayersSlice = players.slice(1..2)
println(upcomingPlayersSlice.joinToString()) // > Bob, Cindy
```

The range you used is `1..2`, which represents the second and third items in the array. You can use an index here as long as the start value is smaller than or equal to the end value and both are within the bounds of the array. If the start value is greater than the end value, the result will be empty.

The object returned from the `slice()` method is a separate array or list from the original, so making modifications to the slice does not affect the original array or list.

Checking for an element

You can check if there's at least one occurrence of a specific element by using the `in` operator, which returns `true` if it finds the element, and `false` otherwise.

You can use this strategy to write a function that checks if a given player is in the game:

```
fun isEliminated(player: String): Boolean {
    return player !in players
}
```

You're using the `!` operator to see if a player is *not* in `players`. Now you can use this function any time you need to check if a player has been eliminated:

```
println(isEliminated("Bob")) // > false
```

The `in` operator corresponds to the `contains()` method. You can test for the existence of an element in a specific range using `slice()` and `contains()` together:

```
players.slice(1..3).contains("Alice") // false
```

Now that you can get data *out* of your arrays and lists, it's time to look at mutable lists and how to change their values.

Modifying lists

You can make all kinds of changes to mutable lists, such as adding and removing elements, updating existing values, and moving elements around into a different order. In this section, you'll see how to work with the list to match up with what's going on in your game.

Appending elements

If new players want to join the game, they need to sign up and add their names to the list. Eli is the first player to join the existing four players.

You can add Eli to the end of the array using the `add()` method:

```
players.add("Eli")
```

If you try to add anything other than a string, the compiler will show an error. Remember, lists can only store values of the same type. Also, `add()` only works with mutable lists.

The next player to join the game is Gina. You can add her to the game another way, by using the `+=` operator:

```
players += "Gina"
```

The right-hand side of this expression is a single element: the string "Gina". By using `+=`, you're adding the element to the end of `players`. Now the list looks like this:

```
println(players.joinToString())  
// > "Alice", "Bob", "Cindy", "Dan", "Eli", "Gina"
```

Here, you added a single element to the array, but you can see how easy it would be to add *multiple* items using the `+=` operator by adding more names after Gina's.

While arrays are of fixed-size, you *can* in fact use the `+=` operator with an array that is declared as `var`.

```
var array = arrayOf(1, 2, 3)  
array += 4  
println(array.joinToString()) // > 1, 2, 3, 4
```

But beware that you are not actually appending the value onto the existing array, but instead creating an entirely new array that has the additional element and assigning the new array to the original variable.

Inserting elements

An unwritten rule of this card game is that the players' names have to be in alphabetical order. This list is missing a player that starts with the letter F. Luckily, Frank has just arrived. You want to add him to the list between Eli and Gina. To do that, you can use a variant of the `add()` method that accepts an index as the first argument:

```
players.add(5, "Frank")
```

The first argument defines where you want to add the element. Remember that the list is zero-indexed, so index 5 is Gina's index, causing her to move up as Frank takes her place.

Removing elements

During the game, the other players caught Cindy and Gina cheating. They should be removed from the game! You can remove them by name using the `remove()` method:

```
val wasPlayerRemoved = players.remove("Gina")
println("It is $wasPlayerRemoved that Gina was removed")
// > It is true that Gina was removed
```

This method does two things: It removes the element and then returns a Boolean indicating whether the removal was successful, so that you can make sure the cheater has been removed!

To remove Cindy from the game, you need to know the exact index where her name is stored. Looking at the list of players, you see that she's third in the list, so her index is 2. You can remove Cindy using `removeAt()`.

```
val removedPlayer = players.removeAt(2)
println("$removedPlayer was removed") // > Cindy was removed
```

Unlike `remove()`, `removeAt()` returns the element that was removed from the list. You could then add that element to a list of cheaters!

But how would you get the index of an element if you didn't already know it? There's a method for that! `indexOf()` returns the *first index* of the element, because the list might contain multiple copies of the same value. If the method doesn't find the element, it returns `-1`.

Mini-exercise

Use `indexOf()` to determine the position of the element "Dan" in `players`.

Updating elements

Frank has decided everyone should call him Franklin from now on. You could remove the value "Frank" from the list and then add "Franklin", but that's too much work for a simple task. Instead, you should use the indexing syntax to update the name.

```
println(players.joinToString())  
// > "Alice", "Bob", "Dan", "Eli", "Frank"  
players[4] = "Franklin"  
println(players.joinToString())  
// > "Alice", "Bob", "Dan", "Eli", "Franklin"
```

Be careful to not use an index beyond the bounds of the list, or your code will crash.

As the game continues, some players are eliminated, and new ones come to replace them. You can use indexing to replace the old players with the new:

```
players[3] = "Anna"  
players.sort()  
println(players.joinToString()) // > "Alice", "Anna", "Bob",  
"Dan", "Franklin"
```

This code replaces the player Eli with the player Anna. You then call `sort()` on the list to make sure the list remains sorted in alphabetical order.

When updating an element, the indexing syntax is equivalent to calling `set()` on the list.

```
players.set(3, "Anna")
```

As IntelliJ IDEA will tell you, using the indexing syntax is generally preferred over using `get()` and `set()` on the collection types.

Note that while arrays are of fixed size and can otherwise not be changed, you *can* update the elements of an array using indexing syntax.

```
val arrayOfInts = arrayOf(1, 2, 3)
arrayOfInts[0] = 4
println(arrayOfInts.toString()) // > 4, 2, 3
```

Iterating through a list

It's getting late, so the players decide to stop for the night and continue tomorrow. In the meantime, you'll keep their scores in a separate list. You'll investigate a better approach for this when you learn about maps, but for now you can continue to use lists:

```
val scores = listOf(2, 2, 8, 6, 1)
```

Before the players leave, you want to print the names of those still in the game. Like for arrays, you can do this using the `for` loop you read about in Chapter 5, “Advanced Control Flow”:

```
for (player in players) {
    println(player)
}
// > Alice
// > Anna
// > Bob
// > Dan
// > Franklin
```

This code goes over all the elements of `players`, from index 0 up to `players.size - 1` and prints their values. In the first iteration, `player` is equal to the first element of the list; in the second iteration, it's equal to the second element of the list; and so on, until the loop has printed all the elements in the list.

If you need the index of each element, you can iterate over the return value of the list's `withIndex()` method, which can be *destructured* to each element's index and value:

```
for ((index, player) in players.withIndex()) {
    println("${index + 1}. $player")
}
// > 1. Alice
// > 2. Anna
// > 3. Bob
```

```
// > 4. Dan  
// > 5. Franklin
```

Now you can use the technique you've just learned to write a function that takes a list of integers as its input and returns the sum of its elements:

```
fun sumOfElements(list: List<Int>): Int {  
    var sum = 0  
    for (number in list) {  
        sum += number  
    }  
    return sum  
}
```

You could use this function to calculate the sum of the players' scores:

```
println(sumOfElements(scores)) // > 19
```

Mini-exercise

Write a for loop that prints the players' names and scores.

Nullability and collection types

When working with arrays, lists, and other collection types, special consideration should be given to nullability. Are the elements of a collection nullable, for example, or is the collection itself nullable?

A nullable list can be created as follows:

```
var nullableList: List<Int>? = listOf(1, 2, 3, 4)
```

The individual elements are of type `Int` and cannot be null, but the list itself *can* be null.

```
nullableList = null
```

On the other hand, you can create a list with elements that are nullable by shifting nullability to the type argument:

```
var listOfNullables: List<Int?> = listOf(1, 2, null, 4)
```


If you try to set the list itself to null, you'll get a compiler error.

```
listOfNullables = null // Error: Null can not be a value of a
non-null type
```

You can go to the extreme with nullability by letting both the list and its elements be null.

```
var nullableListOfNullables: List<Int?>? = listOf(1, 2, null, 4)
nullableListOfNullables = null
```

As with all nullable types, you should always be conscious of when you should allow the collection or its elements to be null.

Challenges

Check out the following challenges to test your knowledge of Kotlin arrays and lists. As always, you can check out the solutions in the materials for this chapter.

1. Which of the following 1-10 are valid statements?

```
1. val array1 = Array<Int>()
2. val array2 = array0f()
3. val array3: Array<String> = array0f()
```

For the next three statements, array4 has been declared as:

```
val array4 = array0f(1, 2, 3)
```

```
4. println(array4[0])
5. println(array4[5])
6. array4[0] = 4
```

For the final five statements, array5 has been declared as:

```
val array5 = array0f(1, 2, 3)
```

```
7. array5[0] = array5[1]
8. array5[0] = "Six"
9. array5 += 6
10. for item in array5 { println(item) }
```

2. Write a function that removes the first occurrence of a given integer from a list of integers. This is the signature of the function:

```
fun removeOne(item: Int, list: List<Int>): List<Int>
```

3. Write a function that removes all occurrences of a given integer from a list of integers. This is the signature of the function:

```
fun remove(item: Int, list: List<Int>): List<Int>
```

4. Arrays and lists have a `reverse()` method that reverses all the elements in-place, that is, within the original array or list. Write a function that does a similar thing, without using `reverse()`, and returns a new array with the elements of the original array in reverse order. This is the signature of the function:

```
fun reverse(array: Array<Int>): Array<Int>
```

5. The function below returns a random number between from (inclusive) and the to (exclusive):

```
import java.util.Random
val random = Random()
fun rand(from: Int, to: Int) : Int {
    return random.nextInt(to - from) + from
}
```

Use it to write a function that shuffles the elements of an array in random order. This is the signature of the function:

```
fun randomized(array: Array<Int>): Array<Int>
```

6. Write a function that calculates the minimum and maximum value in an array of integers. Calculate these values yourself; don't use the methods `min` and `max`. Return `null` if the given array is empty.

This is the signature of the function:

```
fun minMax(numbers: Array<Int>): Pair<Int, Int>?
```

Hint: You can use the `Int.MIN_VALUE` and `Int.MAX_VALUE` constants within the function.

Key points

- **Arrays** are ordered collections of values of the same type.
- There are special classes such as `IntArray` created as arrays of Java primitive types.
- **Lists** are similar to arrays but have the additional feature of being dynamically-sized.
- You can add, remove, update, and insert elements into **mutable lists**.
- Use **indexing** or one of many methods to access and update elements.
- Be wary of accessing an index that's out of bounds.
- You can iterate over the elements of an array or list using a `for` loop or using `forEach`.
- You can check for elements in an array or list using `in`.
- Special consideration should be given when working with nullable lists and lists with nullable elements.

Where to go from here?

Now that you've learned about the array and list collection types in Kotlin, you can now move on to learning about two other common collection types: **maps** and **sets**.